

Documentation complète — installeur OL9.6 custom

Référence exhaustive de chaque fonction, chaque commande système appelée, et chaque constante du projet. Organisé module par module dans l'ordre d'exécution (`main.py`).

Aucune classe dans ce projet : tout est écrit sous forme de fonctions simples. C'est volontaire — c'est un outil de provisioning "one-shot" (on l'exécute une fois par poste, pas d'état à faire vivre entre plusieurs appels), donc pas besoin d'objets avec un cycle de vie. Une fonction qui prend des paramètres et renvoie un résultat est plus simple à lire, à tester isolément, et à documenter qu'une hiérarchie de classes.

Sommaire

1. [config.py](#)
 2. [rpm_installer.py](#)
 3. [accounts.py](#)
 4. [shortcuts.py](#)
 5. [gnome_extensions.py](#)
 6. [opt_permissions.py](#)
 7. [java_setup.py](#)
 8. [password_policy.py](#)
 9. [deploy_bundle.py](#)
 10. [install.py](#)
 11. [main.py](#)
-

config.py

```
load_config(path="config.json")
```

Syntaxe

```
load_config(path="config.json") -> dict
```

Utilité Lit un fichier JSON et le transforme en `dict` Python. C'est le seul point d'entrée vers `config.json` dans tout le projet — aucun autre module ne lit directement ce fichier.

Exemple

```
from config import load_config cfg = load_config() # lit "config.json" dans le dossier
courant cfg = load_config("/etc/mon.json") # ou un chemin explicite
print(cfg["rpm_dir"]) # "/opt/install/rpms"
```

Erreurs à ne pas faire - Appeler `load_config()` depuis un dossier qui n'est pas celui du projet : le chemin par défaut `"config.json"` est **relatif au dossier de travail courant** (`cwd`), pas au dossier du script. Si tu lances `sudo python3 /tmp/scriptInstallApps/main.py` depuis `/home/naval/`, il cherchera `/home/naval/config.json`, pas celui du projet. Toujours lancer les scripts depuis le dossier qui contient `config.json`. - Oublier qu'un JSON malformé (virgule en trop, guillemets simples au lieu de doubles) fait planter `json.load` avec une `json.decoder.JSONDecodeError` peu lisible. Toujours valider le JSON après édition manuelle (ex. `python3 -m json.tool config.json`).

Pourquoi ainsi `json` est dans la bibliothèque standard (aucune dépendance externe à installer sur une machine hors ligne), le format est facile à éditer à la main et à versionner dans git. On a choisi JSON plutôt que YAML précisément pour éviter d'installer PyYAML sur une cible qui n'a pas accès à internet.

`rpm_installer.py`

`find_rpms(rpm_dir)`

Syntaxe

```
find_rpms(rpm_dir: str) -> list[str]
```

Utilité Liste tous les fichiers `*.rpm` présents dans `rpm_dir`. Lève une erreur si le dossier n'existe pas ou s'il est vide, plutôt que de renvoyer silencieusement une liste vide.

Exemple

```
fichiers = find_rpms("/opt/install/rpms") # ->
["/opt/install/rpms/jq-1.6-1.el9.x86_64.rpm", "/opt/install/rpms/xterm-...", ...]
```

Erreurs à ne pas faire - Passer un chemin qui existe mais qui pointe vers un fichier (pas un dossier) : `os.path.isdir` renverra `False` et tu auras un `FileNotFoundError` avec un message "le dossier n'existe pas" un peu trompeur (le fichier existe, ce n'est juste pas un dossier). - Croire que `glob.glob("*.rpm")` est récursif : il ne descend pas dans les sous-dossiers. Si les RPM sont rangés dans des sous-dossiers par catégorie, `find_rpms` ne les verra pas.

Pourquoi ainsi `glob` fait exactement ce qu'on veut ici (motif simple à plat, pas de recherche récursive) sans dépendance externe. Le garde-fou "dossier vide" existe parce qu'un dossier `/opt/install/rpms` vide n'est presque toujours pas une intention volontaire mais un oubli de déploiement (bundle pas encore copié) — mieux vaut planter tout de suite avec un message clair que de laisser `install_rpms([])` s'exécuter silencieusement sur une liste vide.

`calculate_sha256(file_path)`

Syntaxe

```
calculate_sha256(file_path: str) -> str
```

Utilité Calcule l'empreinte SHA256 d'un fichier et la renvoie en hexadécimal (chaîne de 64 caractères), pour vérifier son intégrité.

Exemple

```
empreinte = calculate_sha256("/opt/install/rpms/jq-1.6-1.el9.x86_64.rpm") # ->
"3f786850e387550fdab836ed7e6dc881de23001b3d5b1a5..."
```

Erreurs à ne pas faire - L'ouvrir en mode texte (`open(file_path)` sans `"rb"`) : un RPM est un fichier binaire, l'ouvrir en mode texte casserait l'encodage sur certains octets et donnerait un hash complètement faux (ou une exception `UnicodeDecodeError` selon la plateforme). - Appeler cette fonction sur un très gros fichier en boucle serrée sans réfléchir à la mémoire : `f.read()` charge **tout le fichier en RAM d'un coup**. Pour des RPM (quelques Mo à quelques centaines de Mo), c'est sans risque ; pour des fichiers de plusieurs Go, il faudrait lire par blocs (`f.read(65536)` en boucle avec `hash.update()`).

Pourquoi ainsi `hashlib` est `stdlib`, SHA256 est le standard actuel pour ce genre de vérification (SHA256SUMS est le nom de fichier conventionnel généré par `sha256sum` sous Linux). Lire le fichier entier d'un coup plutôt que par blocs simplifie le code et reste largement acceptable vu la taille des fichiers concernés ici.

```
parse_sha256sums(sums_file)
```

Syntaxe

```
parse_sha256sums(sums_file: str) -> dict[str, str]
```

Utilité Lit un fichier au format `sha256sum` (une ligne par fichier : `empreinte nom_fichier`) et le transforme en dict `{nom_fichier: empreinte}` pour un lookup facile.

Exemple

```
# Contenu de SHA256SUMS : # 3f786850e387550fdab836ed7e6dc881de23001b
jq-1.6-1.el9.x86_64.rpm # a94a8fe5ccb19ba61c4c0873d391e987982fbbd3 xterm-...
empreintes = parse_sha256sums("/opt/install/rpms/SHA256SUMS") # ->
{"jq-1.6-1.el9.x86_64.rpm": "3f786850...", "xterm-...": "a94a8fe5..."}
```

Erreurs à ne pas faire - Utiliser `ligne.split(" ", 1)` au lieu de `ligne.split(None, 1)` : `sha256sum` sépare l'empreinte et le nom de fichier par **deux espaces** (ou un espace + un indicateur de mode), pas un seul. `split(None, 1)` découpe sur n'importe quelle suite d'espaces/tabulations, ce qui gère les deux cas ("`empreinte fichier`" et "`empreinte *fichier`") sans se soucier du nombre exact d'espaces. - Oublier le `.rstrip("*")` : le `*` avant le nom de fichier signifie "mode binaire" chez `sha256sum` — si on ne le retire pas, la clé du dict sera `"*jq-1.6-1.el9.x86_64.rpm"` au lieu de `"jq-1.6-1.el9.x86_64.rpm"`, et `verify_checksums` ne retrouvera jamais la correspondance (tous les fichiers ressortiront comme "absents de SHA256SUMS").

Pourquoi ainsi Un dict donne un lookup en $O(1)$ par nom de fichier dans `verify_checksums`, plutôt que de parcourir une liste à chaque fichier RPM ($O(n)$ par recherche, $O(n^2)$ au total). Vu que le nombre de RPM reste modeste ici, la différence de performance est négligeable, mais c'est la structure de données naturelle pour "je veux retrouver une valeur à partir d'une clé".

```
verify_checksums(rpm_dir, sums_file=None)
```

Syntaxe

```
verify_checksums(rpm_dir: str, sums_file: str | None = None) -> list[str]
```

Utilité Combine les trois fonctions précédentes : liste les RPM, lit SHA256SUMS, compare chaque empreinte calculée à l'empreinte attendue. Renvoie la liste des chemins RPM si tout est bon, lève une `ValueError` sinon.

Exemple

```
rpm_files = verify_checksums("/opt/install/rpms") # -> ["/opt/install/rpms/jq-...",  
"/opt/install/rpms/xterm-..."] si tout correspond  
verify_checksums("/opt/install/rpms") # ValueError: Vérification SHA256 échouée: # -  
jq-1.6-1.el9.x86_64.rpm: empreinte différente (attendu ..., obtenu ...) # - xterm-...:  
absent de SHA256SUMS
```

Erreurs à ne pas faire - Passer un `sums_file` qui ne correspond pas réellement au contenu actuel de `rpm_dir` (ex. un SHA256SUMS généré avant d'avoir ajouté de nouveaux RPM dans le dossier) : la fonction lèvera une erreur "absent de SHA256SUMS" pour chaque nouveau fichier, ce qui est le comportement voulu mais peut surprendre si on a oublié de régénérer le fichier après un ajout. - S'arrêter au premier `raise` en pensant corriger un fichier à la fois : la fonction est justement conçue pour accumuler **tous** les problèmes dans `invalides` avant de lever une seule erreur groupée, pour éviter les allers-retours "je corrige un fichier, je relance, j'en découvre un autre".

Pourquoi ainsi Grouper toutes les erreurs en une seule exception (plutôt que `raise` dès le premier fichier suspect) donne une vue complète du problème en un seul run — précieux sur une machine hors ligne où chaque essai/erreur prend du temps (il faut retransférer des fichiers, relancer le script).

```
install_rpms(rpm_files)
```

Syntaxe

```
install_rpms(rpm_files: list[str]) -> str
```

Utilité Installe la liste de RPM donnée via `dnf`. Lève `PermissionError` si pas root, `RuntimeError` si `dnf` échoue.

Commande système utilisée

```
dnf install -y --disablerepo=* <fichier1.rpm> <fichier2.rpm> ...
```

- `-y` : répond "oui" automatiquement à toute confirmation (le script tourne sans interaction).
- `--disablerepo=*` : désactive **tous** les dépôts configurés (distants). Sans ce flag, une machine avec un accès réseau, même partiel ou lent, ferait attendre `dnf` indéfiniment en essayant de contacter des dépôts injoignables. Avec ce flag, `dnf` ne peut résoudre les dépendances qu'entre les fichiers explicitement passés en argument.

Exemple

```
rpm_files = verify_checksums("/opt/install/rpms") sortie = install_rpms(rpm_files)  
print(sortie) # stdout de dnf : liste des paquets installés, tailles, etc.
```

Erreurs à ne pas faire - Ajouter `--allow-erase` sans comprendre pourquoi `dnf` en a besoin : ce flag autorise `dnf` à **désinstaller** des paquets déjà présents pour résoudre la transaction. Si le dossier `rpm_dir` contient des doublons de composants système (mauvaise version d'une lib déjà installée), `dnf` peut proposer de désinstaller des paquets protégés (`dnf`, `yum` eux-mêmes) — signe qu'il faut nettoyer le lot de RPM, pas forcer le flag. Voir l'incident documenté dans `INSTALL.md`. - Ajouter `--alldeps` au moment du **téléchargement** (`dnf download`) en pensant que c'est équivalent à "prends toutes les dépendances nécessaires" : ce flag force le téléchargement de **toutes** les dépendances, y compris celles déjà satisfaites localement, et va chercher la dernière version disponible sur le dépôt distant — ce qui peut ramener des centaines de paquets système non désirés (`oraclelinux-release` compris). `install_rpms` elle-même n'a pas ce problème (elle ne télécharge rien), mais le bundle qu'on lui donne en amont doit avoir été construit sans `--alldeps`.

Pourquoi ainsi `subprocess.run(..., capture_output=True, text=True)` plutôt que `os.system()` : on récupère `stdout/stderr` en tant que texte pour construire un message d'erreur exploitable, et on évite les problèmes d'injection shell qu'`os.system()` peut introduire si un chemin de fichier contient des caractères spéciaux (`os.system` passe par un shell, `subprocess.run` avec une liste d'arguments non).

accounts.py

`prompt_user_info()`

Syntaxe

```
prompt_user_info() -> dict # -> {"username": str, "password": str, "force_change": bool}
```

Utilité Pose trois questions à l'opérateur dans le terminal (nom d'utilisateur, mot de passe temporaire, forcer le changement au prochain login) et renvoie les réponses sous forme de dict, prêt à être passé à `create_user(**infos)`.

Exemple

```
infos = prompt_user_info() # Nom d'utilisateur : jdupont # Mot de passe temporaire :  
(saisie invisible) # Forcer le changement de mot de passe à la prochaine connexion ?  
(1 = oui, Entrée = non) : 1 print(infos) # {"username": "jdupont", "password": "...",  
"force_change": True}
```

Erreurs à ne pas faire - Afficher `infos["password"]` dans un `print()` de debug : le mot de passe est en clair dans le dict Python en mémoire (c'est nécessaire pour le passer à `chpasswd`), mais il ne doit **jamais** être loggé, imprimé, ou écrit dans un fichier. Le `__main__` de ce module fait exprès de n'afficher que `username` et `force_change`. - Croire que `getpass.getpass()` fonctionne dans un terminal qui n'est pas un vrai TTY (ex. certains IDE, ou un script lancé via un pipe) : dans ce cas, `getpass` bascule sur un `input()` classique **avec** écho visible, avec un avertissement — pas une erreur silencieuse, mais un piège si on ne le sait pas.

Pourquoi ainsi `getpass` plutôt que `input()` pour le mot de passe : `input()` afficherait la saisie en clair à l'écran (visible si quelqu'un regarde par-dessus l'épaule, ou si le terminal est partagé/enregistré). La boucle `while not username` plutôt qu'une validation après coup : on ne peut pas continuer sans nom d'utilisateur, autant bloquer immédiatement plutôt que de laisser l'opérateur remplir tout le formulaire pour échouer à la fin.

```
create_user(username, password, force_change=False)
```

Syntaxe

```
create_user(username: str, password: str, force_change: bool = False) -> None
```

Utilité Crée un compte local Linux : `useradd` (avec tous les défauts système), `chpasswd` (mot de passe initial), et `chage -d 0` si `force_change=True` (le compte devra changer son mot de passe à la prochaine connexion).

Commandes système utilisées

```
useradd <username> chpasswd # avec "username:password\n" sur stdin chage -d 0  
<username> # seulement si force_change=True
```

Exemple

```
create_user("jdupont", "TempPass123", force_change=True) # Crée /home/jdupont, pose le  
mot de passe, force le renouvellement au prochain login.
```

Erreurs à ne pas faire - Passer le mot de passe en argument de `chpasswd` (ex. `["chpasswd", f"{username}:{password}"]`) au lieu de le passer via `input=` (`stdin`) : un argument de commande est visible par n'importe quel utilisateur du système via `ps aux` ou `/proc/<pid>/cmdline` pendant toute la durée d'exécution du process, même bref. Passer par `stdin` évite complètement cette fuite. - Rappeler `create_user` deux fois avec le même `username` : `useradd` échouera avec "l'utilisateur existe déjà" et lèvera une `RuntimeError` — c'est voulu (pas de `silent no-op` qui masquerait une erreur d'opérateur), mais ça veut dire que ce script n'est pas idempotent sur la création de compte, contrairement à `java_setup.copy_java` par exemple.

Pourquoi ainsi Aucune option `useradd` personnalisée (pas de `-m`, `-s`, `-G...`) : on a délibérément gardé les valeurs par défaut du système (décision prise en cours de route avec l'utilisateur) plutôt que d'ajouter de la configurabilité non demandée. `chage -d 0` plutôt que `passwd -e` : les deux forcent un changement de mot de passe au prochain login, mais `chage -d 0` est la commande "propre" pour manipuler uniquement la date du dernier changement sans toucher à d'autres attributs du compte.

shortcuts.py

```
find_desktop_files(src_dir)
```

Syntaxe

```
find_desktop_files(src_dir: str) -> list[str]
```

Utilité Identique dans l'esprit à `find_rpms`, mais pour les fichiers `*.desktop`. Lève une erreur si le dossier n'existe pas ou est vide.

Exemple

```
fichiers = find_desktop_files("/opt/install/desktop") # ->  
["/opt/install/desktop/csdlab.desktop"]
```

Erreurs à ne pas faire - Mettre des fichiers `.desktop` mal formés (sans extension exacte `.desktop`, ex. `monapp.Desktop` avec une majuscule) : `glob.glob("*.desktop")` est sensible à la casse sous Linux, le fichier serait silencieusement ignoré.

Pourquoi ainsi Duplication volontaire avec `find_rpms` plutôt que factorisation en une fonction générique `find_files(dir, pattern)` : les deux fonctions ont des messages d'erreur différents et des usages assez distincts pour que la petite duplication reste plus lisible qu'une abstraction prématurée à deux appelants.

```
copy_desktop_files(src_dir, dest_dir="/usr/share/applications/")
```

Syntaxe

```
copy_desktop_files(src_dir: str, dest_dir: str = "/usr/share/applications/") ->
list[str]
```

Utilité Copie chaque `.desktop` trouvé vers `dest_dir`, puis pose `chmod 644` dessus (lecture pour tout le monde, écriture pour le propriétaire seulement).

Exemple

```
copies = copy_desktop_files("/opt/install/desktop") # ->
["/usr/share/applications/cslab.desktop"]
```

Erreurs à ne pas faire - Utiliser `shutil.copy2` au lieu de `shutil.copy` en pensant "plus complet donc mieux" : `copy2` copie aussi les métadonnées (dates, permissions) du fichier source. Ici on **veut** poser nous-mêmes les permissions (`0o644`) après coup, donc hériter des permissions de la source (potentiellement `600`, ou `777` selon comment le fichier a été préparé) serait contre-productif. - Oublier que `dest_dir` doit déjà exister : `shutil.copy` ne crée pas les dossiers intermédiaires (contrairement à `os.makedirs(..., exist_ok=True)`). Sur une cible standard, `/usr/share/applications/` existe toujours (fourni par le paquet `filesystem`), donc ce n'est pas un problème en pratique, mais ça le serait avec un `dest_dir` personnalisé qui n'existe pas encore.

Pourquoi ainsi `chmod 644` explicite après la copie plutôt que de compter sur un `umask` bien configuré : le `umask` de l'utilisateur qui lance le script (root, via `sudo`) peut varier d'une machine à l'autre ou d'une session à l'autre — poser la permission explicitement rend le résultat prévisible quel que soit le contexte d'exécution.

gnome_extensions.py

```
ensure_dconf_profile(profile_path="/etc/dconf/profile/user")
```

Syntaxe

```
ensure_dconf_profile(profile_path: str = "/etc/dconf/profile/user") -> None
```

Utilité Garantit que le profil dconf "user" contient bien la ligne `system-db: local`, sans laquelle les réglages système qu'on écrit ensuite (via `write_extensions_profile`) seraient tout simplement ignorés par dconf. Ne réécrit rien si c'est déjà en place.

Exemple

```
ensure_dconf_profile() # Crée /etc/dconf/profile/user avec : # user-db:user #
system-db:local # (ou ne fait rien si déjà présent)
```

Erreurs à ne pas faire - Écraser ce fichier sans vérifier son contenu existant s'il a déjà été personnalisé (ex. un admin a ajouté un `system-db:` supplémentaire pour une autre politique) : la fonction vérifie juste la présence de la sous-chaîne `"system-db:local"` avant de réécrire — si le fichier existe déjà avec ce contenu, il n'est pas touché ; s'il existe avec un contenu **différent** qui ne contient pas cette ligne, il sera **totalemtent écrasé** (pas fusionné). À garder en tête si ce fichier est aussi géré par un autre outil.

Pourquoi ainsi Vérifier avant d'écrire (plutôt que d'écraser systématiquement) rend la fonction idempotente et sûre à rappeler à chaque run de `main.py` sans effet de bord si rien n'a changé.

```
write_extensions_profile(extension_uuids, db_dir="/etc/dconf/db/local.d")
```

Syntaxe

```
write_extensions_profile(extension_uuids: list[str], db_dir: str =
"/etc/dconf/db/local.d") -> str
```

Utilité Écrit le fichier `00-extensions` dans `db_dir` avec la clé `enabled-extensions` au format GVariant attendu par dconf. Renvoie le chemin du fichier écrit.

Exemple

```
chemin = write_extensions_profile(["dash-to-dock@micxgx.gmail.com",
"appindicatorsupport@rgcjonas.gmail.com"]) # Écrit /etc/dconf/db/local.d/00-extensions
: # [org/gnome/shell] # enabled-extensions=['dash-to-dock@micxgx.gmail.com',
'appindicatorsupport@rgcjonas.gmail.com']
```

Erreurs à ne pas faire - Donner un UUID d'extension incorrect (le vrai UUID GNOME, pas le nom affiché dans l'interface) : dconf acceptera n'importe quelle chaîne sans erreur, mais l'extension ne s'activera jamais puisque le nom ne correspondra à rien d'installé. Toujours vérifier l'UUID exact via `gnome-extensions list` sur une machine où l'extension est déjà installée. - Appeler cette fonction seule sans avoir appelé `ensure_dconf_profile` avant (ou sans avoir fait `apply_dconf` après) : le fichier serait écrit mais totalement sans effet.

Pourquoi ainsi Le nom de fichier `00-extensions` (avec le préfixe numérique `00-`) suit la convention dconf : les fichiers dans `db/local.d/` sont fusionnés dans l'ordre alphabétique, le préfixe numérique sert à contrôler cet ordre si jamais d'autres fichiers de config sont ajoutés plus tard dans ce dossier.

```
apply_dconf()
```

Syntaxe

```
apply_dconf() -> None
```

Utilité Compile les fichiers texte de `/etc/dconf/db/local.d/` en base binaire dconf. Sans cet appel, les fichiers écrits par `write_extensions_profile` restent de simples fichiers texte inertes.

Commande système utilisée


```
dconf update
```

Exemple

```
apply_dconf() # Après ça, les nouvelles valeurs par défaut sont actives pour toute nouvelle session.
```

Erreurs à ne pas faire - Oublier cet appel après avoir modifié un fichier dans `db/local.d/` à la main (en dehors du script) : c'est l'erreur classique avec `dconf`, "j'ai édité le fichier mais rien ne change" — `dconf` ne relit jamais les fichiers texte directement, seulement sa base binaire compilée.

Pourquoi ainsi Fonction séparée plutôt que fusionnée dans `enable_extensions` : ça permet de rappeler `dconf update` isolément si on modifie `db/local.d/` par un autre moyen (à la main, ou un autre script) sans avoir à repasser par toute la logique d'`enable_extensions`.

```
enable_extensions(extension_uuids)
```

Syntaxe

```
enable_extensions(extension_uuids: list[str]) -> None
```

Utilité Fonction "chef d'orchestre" du module : enchaîne `ensure_dconf_profile` → `write_extensions_profile` → `apply_dconf`. C'est la seule fonction de ce module appelée depuis `main.py`.

Exemple

```
enable_extensions(["dash-to-dock@micxgx.gmail.com"])
```

Erreurs à ne pas faire - Appeler les fonctions internes (`ensure_dconf_profile`, `write_extensions_profile`, `apply_dconf`) directement depuis `main.py` au lieu de passer par `enable_extensions` : ça marche techniquement, mais ça duplique l'ordre d'appel dans deux fichiers différents — si l'ordre doit changer un jour, il faudrait penser à le corriger aux deux endroits.

Pourquoi ainsi Une seule vérification `os.geteuid() != 0` ici, pas dans chacune des trois sous-fonctions : elles écrivent toutes dans des chemins système protégés, donc la vérification `root` au niveau de la fonction publique suffit ; la dupliquer trois fois n'ajouterait rien.

opt_permissions.py

```
set_opt_permissions(app_dir)
```

Syntaxe

```
set_opt_permissions(app_dir: str) -> None
```

Utilité Pose `chmod 755` sur `app_dir` lui-même, puis récursivement sur tous les sous-dossiers **et** fichiers en dessous.

Exemple

```
set_opt_permissions("/opt/monapp") # /opt/monapp devient 755, et tout ce qu'il
contient (fichiers compris) aussi.
```

Erreurs à ne pas faire - Utiliser cette fonction sur un dossier contenant des fichiers sensibles avec des permissions volontairement restrictives (clés privées, fichiers de config avec des secrets en 600) : la fonction est **récursive sur tout, fichiers compris** (décision prise explicitement avec l'utilisateur, voir la conversation du 2026-07 sur /opt). Un fichier en 600 dans l'arborescence passerait en 755, donc lisible/exécutable par tout le monde. À réserver aux dossiers d'applications sans secret dedans. - Croire que 755 sur un fichier de données (pas un binaire) pose un souci de sécurité : ça rend le fichier "exécutable" au sens des bits Unix, ce qui est inoffensif pour un fichier texte (le bit x est juste ignoré s'il n'y a rien à exécuter), mais reste inhabituel et peut fausser des heuristiques d'outils tiers qui se fient au bit exécutable pour deviner le type de fichier.

Pourquoi ainsi `os.walk` plutôt que `pathlib.Path.rglob("**")` : les deux fonctionnent, mais `os.walk` donne directement dossiers et fichiers séparés à chaque niveau, ce qui évite d'avoir à tester `.is_dir()/is_file()` sur chaque élément — plus direct pour ce cas d'usage précis (poser un `chmod` différent... ici identique, mais la structure reste plus lisible).

java_setup.py

```
copy_java(java_src_dir, version_dirname, install_root="/opt/java")
```

Syntaxe

```
copy_java(java_src_dir: str, version_dirname: str, install_root: str = "/opt/java") ->
str # -> chemin de destination, ex. "/opt/java/8.0.392-tem"
```

Utilité Copie récursivement `java_src_dir/version_dirname` (le JDK extrait) vers `install_root/version_dirname`. Renvoie le chemin final (= le futur `JAVA_HOME`).

Exemple

```
java_home = copy_java("/opt/install/java", "8.0.392-tem") # -> "/opt/java/8.0.392-tem"
```

Erreurs à ne pas faire - Construire ce dossier source sur un partage réseau qui ne supporte pas les liens symboliques (ex. un partage VMware hgfs) : un JDK contient des symlinks (`libjsig.so`, des pages de man localisées...), et `shutil.copypath` échouera à les recréer si le système de fichiers source ou destination ne les supporte pas. Voir l'incident documenté dans `INSTALL.md` — toujours construire/extraire ce genre d'arborescence sur un système de fichiers Linux natif. - Relancer `copy_java` en pensant qu'elle écrasera proprement une installation précédente corrompue : `dirs_exist_ok=True` **fusionne** avec ce qui existe déjà (les fichiers du même nom sont écrasés, mais les fichiers en trop qui ne sont plus dans la source ne sont **pas** supprimés). Si tu changes de version de JDK sans changer `version_dirname`, il faut supprimer `install_root/version_dirname` toi-même avant de relancer.

Pourquoi ainsi `dirs_exist_ok=True` (plutôt que planter si le dossier existe déjà) permet de relancer `main.py` sur une machine déjà partiellement provisionnée sans avoir à nettoyer `/opt/java` à la main entre deux essais.

```
configure_java_env(java_home, profile_path="/etc/profile.d/java.sh")
```

Syntaxe

```
configure_java_env(java_home: str, profile_path: str = "/etc/profile.d/java.sh") ->
None
```

Utilité Écrit un script shell dans `/etc/profile.d/` qui exporte `JAVA_HOME` et ajoute `$JAVA_HOME/bin` en tête du `PATH`. Pose `chmod 644` dessus.

Exemple

```
configure_java_env("/opt/java/8.0.392-tem") # Écrit /etc/profile.d/java.sh : # export
JAVA_HOME=/opt/java/8.0.392-tem # export PATH=$JAVA_HOME/bin:$PATH
```

Erreurs à ne pas faire - S'attendre à ce que `java -version` fonctionne immédiatement dans le terminal **courant** juste après avoir lancé `main.py : /etc/profile.d/*.sh` n'est lu qu'au **login** d'une session shell (login shell). Un terminal déjà ouvert avant le run du script ne verra pas la variable tant qu'on n'a pas ouvert une nouvelle session (ou fait `source /etc/profile.d/java.sh` manuellement dans ce terminal précis). - Ajouter un `source /etc/profile.d/java.sh` dans le script Python en pensant que ça propagera la variable aux futures sessions des utilisateurs : un `subprocess.run(["source", ...])` (ou même un appel shell) ne changerait que l'environnement du sous-process lancé, jamais celui des sessions d'autres utilisateurs, présentes ou futures. C'est un piège classique en scripting shell/Python : chaque process a son propre environnement, hérité par ses enfants mais jamais partagé "vers le haut" ou "sur le côté".

Pourquoi ainsi `/etc/profile.d/` est l'emplacement standard pour ce genre de réglage global sous Linux (lu automatiquement par `bash`, `sh`, etc. au login), plutôt que d'éditer `/etc/environment` ou le `.bashrc` de chaque utilisateur — un seul fichier à gérer, appliqué à tous les comptes présents et futurs sans intervention supplémentaire.

```
install_java(java_src_dir, version_dirname, install_root="/opt/java")
```

Syntaxe

```
install_java(java_src_dir: str, version_dirname: str, install_root: str = "/opt/java")
-> None
```

Utilité Fonction "chef d'orchestre" : `copy_java` → `opt_permissions.set_opt_permissions` (`chmod 755` récursif, réutilisé depuis le module 6) → `configure_java_env`. Seule fonction du module appelée depuis `main.py`.

Exemple

```
install_java("/opt/install/java", "8.0.392-tem")
```

Erreurs à ne pas faire - Appeler `configure_java_env` avant `set_opt_permissions` si jamais tu réorganises ce code : l'ordre actuel (`copie` → `permissions` → `variables d'env`) n'a pas de dépendance stricte entre `permissions` et `variables d'env`, mais il est cohérent avec la logique "d'abord les fichiers sont en place et lisibles, ensuite on les expose à l'environnement".

Pourquoi ainsi Réutilisation de `opt_permissions.set_opt_permissions` plutôt que de dupliquer un `chmod -R 755` ici : c'est exactement la même opération que pour les autres dossiers d'applis sous `/opt/`,

pas de raison d'avoir deux implémentations du même comportement.

password_policy.py

Constante REGLAGES

Syntaxe

```
REGLAGES: dict[str, int] = { "minlen": 5, "difok": 0, "dcredit": 0, "ucredit": 0,
"lcredit": 0, "ocredit": 0, "minclass": 0, "maxrepeat": 0, "maxclassrepeat": 0,
"maxsequence": 0, "usercheck": 0, }
```

Utilité Valeurs par défaut appliquées à `/etc/security/pwquality.conf` : longueur minimale 5 caractères, toutes les autres contraintes de complexité désactivées (0 = pas de restriction), et `usercheck=0` pour autoriser le nom d'utilisateur comme mot de passe.

Erreurs à ne pas faire - Modifier ce dict en pensant que ça n'affecte qu'un run particulier : c'est une constante au niveau du module, partagée par tous les appels à `relax_password_policy()` qui ne précisent pas explicitement `reglages=...`. Pour un réglage ponctuel différent, passer un dict en paramètre plutôt que modifier `REGLAGES` en mémoire.

Pourquoi ainsi Toutes les valeurs à 0 plutôt que de simplement supprimer les lignes du fichier : `pwquality.conf` a ses propres valeurs par défaut internes si une clé est absente (souvent restrictives), donc "absent" ne veut pas dire "désactivé" — il faut explicitement écrire 0 pour être sûr du résultat.

```
relax_password_policy(conf_path="/etc/security/pwquality.conf", reglages=None)
```

Syntaxe

```
relax_password_policy(conf_path: str = "/etc/security/pwquality.conf", reglages: dict
| None = None) -> None
```

Utilité Modifie `conf_path` ligne par ligne : pour chaque clé de `reglages` (ou `REGLAGES` par défaut), remplace la ligne existante si elle est trouvée (commentée ou active), sinon l'ajoute à la fin du fichier. Ne touche à aucune autre ligne.

Exemple

```
relax_password_policy() # Avant : "# minlen = 8" # Après : "minlen = 5" # (les lignes
non concernées, ex. des commentaires explicatifs, restent inchangées)
```

Erreurs à ne pas faire - S'attendre à ce que cette fonction commente/décommente intelligemment : elle ne fait que repérer la clé via une regex (`^\s*#?\s*(\w+)\s*=`) et remplacer **toute la ligne** par `clé = valeur` sans le `#`. Si le fichier avait plusieurs lignes pour la même clé (une commentée, une active, ce qui est rare mais possible après des éditions manuelles successives), seule la **dernière rencontrée dans le fichier** aura le dernier mot (chaque itération de la boucle écrase l'entrée précédente à cette clé dans `lignes[i]`, mais toutes les lignes correspondantes sont réécrites - donc en pratique toutes finissent avec la bonne valeur, pas de bug, juste plusieurs lignes actives identiques dans le fichier final). - Lancer cette fonction sans avoir vérifié qu'elle tourne bien en root : elle écrit dans `/etc/security/`, un dossier protégé par défaut

sur toute distribution Linux.

Pourquoi ainsi Édition ligne par ligne avec une regex plutôt qu'un écrasement complet du fichier : `pwquality.conf` contient souvent des commentaires explicatifs utiles (documentation inline fournie par le paquet `libpwquality`) qu'on veut préserver, seules les lignes de réglage nous intéressent.

deploy_bundle.py

Constante `DESTINATIONS`

Syntaxe

```
DESTINATIONS: dict[str, str] = { "rpms": "/opt/install/rpms", "desktop":  
    "/opt/install/desktop", "java": "/opt/install/java", }
```

Utilité Fait correspondre chaque sous-dossier attendu dans l'archive (`opt/install/<clé>/`) à son dossier de destination final sur la machine cible.

Pourquoi ainsi Un dict plutôt que trois variables séparées (`RPMS_DEST`, `DESKTOP_DEST`, `JAVA_DEST`) : `deploy_bundle` peut boucler dessus (`for nom_dossier, destination in DESTINATIONS.items()`) au lieu de répéter trois fois le même bloc de code pour chaque dossier.

`deploy_bundle(archive_path)`

Syntaxe

```
deploy_bundle(archive_path: str) -> None
```

Utilité Extrait l'archive tar (compressée ou non, peu importe l'extension) dans un dossier temporaire, puis pour chacun des trois sous-dossiers de `DESTINATIONS` : supprime la destination si elle existe déjà, puis copie fraîchement depuis l'extraction. Nettoie le dossier temporaire à la fin, même en cas d'erreur.

Exemple

```
deploy_bundle("/root/bundle_install.tar.gz") # opt/install/rpms/ -> /opt/install/rpms  
# opt/install/desktop/ -> /opt/install/desktop # opt/install/java/ ->  
/opt/install/java
```

Erreurs à ne pas faire - Interrompre le script (Ctrl+C) pendant l'extraction ou la copie : le `finally` garantit que le dossier temporaire est nettoyé, mais si l'interruption tombe pile pendant un `shutil.rmtree(destination)` (juste après avoir vidé une destination, avant d'avoir fini la recopie), tu peux te retrouver avec un dossier de destination à moitié vide. Il vaut mieux relancer `deploy_bundle` en entier dans ce cas plutôt que de bricoler à la main. - Donner une archive dont la structure interne ne commence pas par `opt/install/` (ex. une archive créée avec `tar czf bundle.tar.gz rpms/ desktop/ java/` sans le préfixe `opt/install/`) : la fonction lèvera un `FileNotFoundError` explicite plutôt que d'échouer silencieusement, mais il faut reconstruire l'archive avec le bon préfixe (voir `INSTALL.md`, section construction du bundle).

Pourquoi ainsi Vider la destination avant de recopier (`shutil.rmtree` puis `shutil.copytree`) plutôt que de fusionner (`dirs_exist_ok=True` comme dans `java_setup.copy_java`) : ici, contrairement au JDK, on

veut activement **empêcher** un mélange entre un ancien bundle RPM et un nouveau — c'est la leçon tirée de l'incident où un vieux lot de 518 RPM traînait à côté d'un nouveau lot propre et provoquait des conflits `dnf` à répétition.

install.py

`install(archive_path)`

Syntaxe

```
install(archive_path: str) -> None
```

Utilité Point d'entrée unique du projet : appelle `deploy_bundle(archive_path)` puis `main()` (importé depuis `main.py`). Évite d'avoir à lancer deux commandes séparées.

Exemple

```
install("/root/bundle_install.tar.gz") # Équivalent à : # sudo python3
deploy_bundle.py /root/bundle_install.tar.gz # sudo python3 main.py # ...mais en un
seul appel.
```

Erreurs à ne pas faire - Importer `main` avant que `deploy_bundle` ait tourné en pensant gagner du temps (ex. paralléliser les deux) : l'import Python (`from main import main as run_install`) n'exécute que les définitions du module, pas de souci à ce niveau — mais l'**appel** `run_install()` doit impérativement se faire après `deploy_bundle(archive_path)`, sinon `main()` lira des fichiers RPM/desktop/java qui n'ont pas encore été placés par le déploiement.

Pourquoi ainsi Un alias `from main import main as run_install` plutôt qu'un simple `from main import main` : évite toute ambiguïté de lecture entre la fonction `main()` importée et la fonction `install()` définie dans ce même fichier — surtout utile si quelqu'un lit ce fichier sans avoir `main.py` ouvert à côté.

main.py

`main()`

Syntaxe

```
main() -> None
```

Utilité L'orchestrateur final : charge `config.json`, puis enchaîne dans l'ordre RPM → Java → politique de mot de passe → comptes (interactif) → shortcuts → extensions GNOME → permissions /opt.

Exemple

```
# Lancé directement : # sudo python3 main.py # Ou importé et appelé (comme le fait
install.py) : from main import main main()
```

Erreurs à ne pas faire - Modifier l'ordre des appels sans réfléchir aux dépendances implicites entre étapes : par exemple, `password_policy.relax_password_policy()` doit tourner **avant** que les comptes ne changent leur mot de passe eux-mêmes (pas avant `create_user`, qui pose le mot de passe initial via `chpasswd` — celui-là n'est pas soumis à `pwquality` de toute façon — mais avant que l'utilisateur ne se connecte et change son mot de passe forcé). - Lancer `main()` sans être root : chaque sous-fonction vérifie individuellement `os.geteuid() != 0` et lèvera une `PermissionError` à la toute première étape concernée (`install_rpms`), donc l'échec sera rapide et clair, mais autant lancer directement avec `sudo` pour ne pas perdre de temps sur un run partiel.

Pourquoi ainsi Chaque étape est un appel direct à la fonction "chef d'orchestre" du module concerné (`install_rpms`, `install_java`, `enable_extensions...`), jamais aux fonctions internes de bas niveau : `main.py` n'a pas besoin de connaître le détail d'implémentation de chaque module, seulement son point d'entrée public — c'est ce qui permet de faire évoluer un module (ex. changer complètement la façon dont `gnome_extensions.py` active les extensions) sans toucher à `main.py`.